

Linking Google Colab to GitHub repository

Free plan · Upgrade

Share

Resolved to explain concepts before implementing code

This is the natural next step from 1.1 — instead of one straight line, you're now stacking a small network that can bend and curve. Let's break down what's being asked and the concepts behind each piece.

The big picture first

In 1.1 you learned that $\hat{y} = w \cdot x + b$ can only ever be a straight line — no matter how you train it, it can't curve to match sine. This part fixes that by:

1. Adding a **hidden layer** (8 neurons instead of 1) — more "flexibility" in the model
2. Adding **tanh**, a *non-linear* activation function, which is the actual missing ingredient that lets the network bend and curve instead of staying flat/straight

The exercise also has you remove **tanh** at the end to *prove* to yourself that depth alone isn't enough — non-linearity is what actually matters.

Part 1: Initializing parameters

You now have two layers of weights instead of one:

- **w1**, **b1** — transform your single input x ↓ 8 hidden values
- **w2**, **b2** — transform those 8 hidden values back down into 1 output

Write a message...

+

Sonnet 5 Medium



Linking Google Colab to GitHub repository

Free plan · Upgrade

Share

The toy problem: learn the function $y = \sin(x)$ on the interval $[-3, 3]$ from 100 noisy samples. It is 1-dimensional, so we can plot everything — the data, the model's prediction, and the loss going down.

Part 1.1 — A single neuron is just a line

A neuron computes $\hat{y} = w * x + b$. Fit it to the sine data with manual gradient descent.

now the real work starts

```
# TODO: Generate the toy data # x = np.linspace(-3, 3, 100); y = np.sin(x) +
small Gaussian noise (std=0.1) # Plot it. # TODO: Initialise w and b to small
random values. # TODO: Implement ONE gradient-descent step by hand,
then loop it for ~200 steps: # y_hat = w * x + b # loss = np.mean((y_hat - y)
** 2) # MSE # dw = np.mean(2 * (y_hat - y) * x) # dLoss/dw
(derive this on paper!) # db = np.mean(2 * (y_hat - y)) # dLoss/db #
w -= lr * dw # b -= lr * db # Record the loss at every step. # TODO: Plot
(a) the loss curve over the 200 steps, and # (b) the data with the fitted
line on top.
```

walk me trough the requirements for this first part what I am required to do and how i should do it do not explicitly give me the answer just guide me

Show less



Write a message...



Sonnet 5 Medium



Let's break down Part 1.1 into what you actually need to produce and the thinking behind each piece.

1. Generate the toy data

You need 100 x-values evenly spaced from -3 to 3, then compute `sin(x)` for each, and add a bit of random noise on top so it looks like real, imperfect data rather than a perfect curve. Think about:

- What NumPy function gives you evenly-spaced points over a range? (You've got a comment hinting at `np.linspace`.)
- How do you generate Gaussian noise with a specific standard deviation (0.1) in NumPy? There's a function for drawing from a normal distribution.
- Once you have `x` and noisy `y`, a simple scatter plot (matplotlib) confirms your data actually looks like a noisy sine wave before you go further.

2. Initialize `w` and `b`

A single neuron here is literally the equation of a line: `y_hat = w*x + b`. You don't want to start at exactly zero (bad practice, though it w technically still work for this simple case) — start with small random numbers. Ask yourself: what NumPy function draws

Write a message...

+

Sonnet 5 Medium



3. Manual gradient descent loop

This is the core of the exercise — you're doing by hand what PyTorch normally automates. For ~200 iterations, you repeat four things every step:

- **Forward pass:** compute what your current `w` and `b` predict — pretty simple, it's the line equation.
- **Loss:** measure how wrong those predictions are using Mean Squared Error, which the comments already give you.
- **Gradients:** this is the part worth understanding, not just copying — the formulas for `dw` and `db` come from taking the derivative of the MSE loss with respect to `w` and `b`. The comment tells you to "derive this on paper" — genuinely do that with pen and paper first; it'll make the formulas make sense instead of being magic symbols.
- **Update:** nudge `w` and `b` a small step in the direction that reduces the loss — that's what the learning rate (`lr`) and the minus-sign updates are doing.

You'll also need to **store the loss value each iteration** in a list so you can plot it afterward — think about where that list needs to be created (before the loop starts) and where you append to it (inside the loop).



4. Plotting

Write a message...



Sonnet 5 Medium ▾



Linking Google Colab to GitHub repository ▾

Free plan · Upgrade ×

Share

think about where that list needs to be created (before the loop starts) and where you append to it (inside the loop).

4. Plotting

Two plots are expected:

- **Loss curve:** iteration number on x-axis, loss value on y-axis — this should trend downward if gradient descent is working.
- **Fitted line over data:** your original noisy scatter data, with a straight line overlaid using your final `w` and `b` — this shows visually how well a *single straight line* fits sine data (spoiler: not very well, which sets up why deeper networks come next in the lab).

A good gut-check before moving on: does your loss curve actually decrease and flatten out? If it's flat from the start or exploding upward, your learning rate or gradient formulas need a look.

Want to try writing the data-generation part first and share it here so I can check your approach before you move to the gradient descent loop?

📄 🔊 🗨️ 🔄

Write a message...

+

Sonnet 5 Medium ▾



```
# TODO: Generate the toy data
x = np.linspace(-3, 3, 100)
y = np.sin(x) + np.random.normal(scale = 0.1) #small Gaussian
noise (std=0.1)
plt.plot(x, y, 'o', label='toy data')
# Plot it.
plt.show() this is how i did the first part is it correct
```

Identified broadcasting issue with noise generation and plotting validity

You're close, and the structure is right — but there's one subtle bug worth catching yourself.
Look closely at this line:

python

```
y = np.sin(x) + np.random.normal(scale=0.1)
```

Question to think through: `np.random.normal(scale=0.1)` — without telling it how

Write a message...

+

Sonnet 5 Medium



Linking Google Colab to GitHub repository

Free plan · Upgrade

Share

```
python  
  
y = np.sin(x) + np.random.normal(scale=0.1)
```

Question to think through: `np.random.normal(scale=0.1)` — without telling it how many numbers to generate — how many random values does it actually return? Try running just that line by itself in a cell and print the result. Compare that to how many values are in `x` (you generated 100 of them with `linspace`).

If `np.random.normal(scale=0.1)` only gives you **one single number**, what happens when you add that one number to your entire 100-element `x / sin(x)` array? Every single data point would get shifted by the *exact same* noise value — which isn't really "independent Gaussian noise per sample," it's just the whole curve shifted up or down by one random amount.

What you want is 100 *different* random noise values, one added to each of the 100 points. Check NumPy's docs (or your own memory) for how `np.random.normal` lets you specify how many samples to draw — there's a parameter for that beyond just `scale`.

Once you fix that, re-run and look at your plot ↓ does it look like scattered noisy points around a sine curve, or does it look like a perfect, smooth sine wave just shifted up/down?

Write a message...

+

Sonnet 5 Medium



Linking Google Colab to GitHub repository

Free plan · Upgrade

Share

Part 1.2 — Hidden layers and activations: why depth helps

Now build a tiny network by hand: 1 input -> a hidden layer of 8 neurons with `tanh` -> 1 output. Still NumPy, still manual gradients

```
# TODO: Initialise parameters # W1: shape (1, 8), b1: shape (8,) # W2: shape (8, 1), b2: shape (1,) # Small random values (e.g. np.random.randn(...) * 0.5) #
TODO: Forward pass (keep the intermediate values — you need them for
backprop): # z1 = x @ W1 + b1 # pre-activation, shape (100, 8) # h =
np.tanh(z1) # activation # y_hat = h @ W2 + b2 # output, shape (100, 1) #
loss = np.mean((y_hat - y) ** 2) # TODO: Backward pass (the chain rule, layer
by layer): # d_yhat = 2 * (y_hat - y) / len(y) # dW2 = h.T @ d_yhat; db2 =
d_yhat.sum(axis=0) # dh = d_yhat @ W2.T # dz1 = dh * (1 - np.tanh(z1) ** 2)
# derivative of tanh # dW1 = x.T @ dz1; db1 = dz1.sum(axis=0) # TODO:
Update all parameters with learning rate lr, loop for ~3000 steps, # record
the loss. # TODO: Plot the loss curve AND the final fit over the data. # Then
re-run with the tanh REMOVED (h = z1). What happens to the fit?
```

now walk me through this one as well what I am supposed to how I should go about it, and what the activity required me to understand

Show less



16:15

Write a message...



Sonnet 5 Medium

